

HealthCentral

Privacy-First Health Records Assistant

A study of how agentic AI tools is being adopted in software engineering and tech adjacent fields through progress of a passion project

Course:	CS4610 — Senior Project (Part 1)
Institution:	Cal Poly Pomona, Department of Computer Science
Student:	Danny Tran
Project period:	January 2026 – May 2026
Submission date:	May 15 th , 2026
Repository:	github.com/BrooklynD23/HealthCentral

Table of Contents

Table of Contents.....	2
1. Executive Summary	3
2. Motivation	3
3. Project Overview: HealthCentral	4
3.1 What the system does	4
3.2 Architecture	4
3.3 Privacy posture.....	4
4. The Research Thread: Why Frameworks Matter	5
5. Frameworks, Tools, and Techniques Studied.....	5
5.1 Prompting evolution: vibe coding → meta-prompting → context engineering → harness engineering	5
5.2 Claude Code primitives: Skills, Subagents, Hooks, Slash Commands, Plugins, Agent SDK	6
5.3 Autonomous loops: Ralph, GSD v1/v2, Superpowers, everything-claude-code, NemoClaw/OpenClaw	6
5.4 MCP and the tool ecosystem	7
5.5 Models.....	7
6. Chronological Timeline.....	8
7. How These Tools Shaped HealthCentral	9
7.1 Adversarial multi-model review	9
7.2 Skills and hooks as privacy enforcement	10
7.3 Autonomous loops for RAG evaluation sweeps	10
8. Reflections and Lessons	10
9. Deliverables	11
10. References.....	11
Appendix A — Glossary	12

1. Executive Summary

HealthCentral is a privacy-first health records assistant that helps patients understand their own medical lab results without sending data to the cloud. It ingests lab PDFs, extracts structured biomarker data, visualizes trends over time, and uses a locally hosted large language model grounded in Retrieval-Augmented Generation (RAG) to explain results with citations. Every byte of patient data is stored in an AES-256-encrypted SQLCipher database with per-document keys and initialization vectors.

The motivation was something personal: watching my mother navigate an autoimmune condition with almost no explanation of the complex bloodwork she received every few months. The pattern she fell into: Googling terms into a spiral of health anxiety, or waiting weeks to ask a doctor a basic question reflects millions of patients affected, especially non-English speakers, elderly patients managing multiple conditions, and communities with limited specialist access.

But HealthCentral is also a research vehicle. From January through April 2026, the project rode a wave of rapidly evolving agentic AI tools: vibe coding, meta-prompting, context engineering, harness engineering, Claude Code Skills and Subagents, Hooks, Plugins, Ralph-style autonomous loops, the GSD pattern, cross-vendor adversarial review with Codex, Model Context Protocol (MCP) servers for Supabase, Vercel, Serena, and Tavily, and UI scaffolding with Google Stitch. Every one of these landed in the project as it appeared in the broader ecosystem, and HealthCentral's changing architecture is the evidence that each layer actually mattered.

This report explains what the project is, traces the agentic-tool progression from plain prompting to autonomous harnesses, and reflects on what this shift means for the future of software engineering.

2. Motivation

Every few months my mother would come home with a new lab report. CBC panels, inflammation markers, thyroid levels. The numbers she could not interpret and a clinician whose fifteen-minute appointment was already rushing. Her options were to search the internet and read ten pages of contradictory advice, or to wait until the next appointment to ask a basic question. Neither option was good.

Unfortunately, this is not an uncommon problem. Tens of millions of patients in the United States manage chronic conditions that require regular blood work. People whose first language is not English, elderly patients juggling multiple prescriptions, and residents of underserved areas with no nearby specialist all face the same literacy gap.

The computer-science thesis inside HealthCentral is simple: if local models are increasingly becoming good enough, and if agentic tools lower the barrier to building production software, a

single senior-project student should be able to produce something that helps someone. The rest of this report is the evidence that they can.

3. Project Overview: HealthCentral

3.1 What the system does

HealthCentral is a local-first desktop application that turns a folder of lab-report PDFs or images into a longitudinal view of a patient's health. Core capabilities:

- Structured extraction from unstructured PDFs, with analyte synonym mapping and unit normalization so that “WBC”, “white blood cells”, and “leukocyte count” resolve to the same metric with consistent units.
- Visualization of biomarker trends over time — CBC, inflammation markers, thyroid hormones, lipid panel, liver and kidney function — with reference ranges overlaid.
- A locally hosted AI assistant that explains results using Retrieval-Augmented Generation grounded in the patient's own reports, with citations back to the source document and page.
- Doctor-ready summaries: the system can generate a concise printable brief, so the patient arrives at their next appointment with specific, informed questions.
- A human-in-the-loop verification workflow where the user corrects AI extractions, and those corrections feed back into synonym maps and unit normalizers, so accuracy improves over time for that specific patient.

3.2 Architecture

HealthCentral is split into a FastAPI (Python) backend and a React + TypeScript frontend. Backend services are async where it helps: PDF ingestion, embedding generation, and search are all non-blocking. The data layer is Postgres-compatible via SQLCipher-wrapped SQLite for single-user local deployment, with per-document encryption keys and unique initialization vectors so that even a compromised backup file leaks nothing without the master key. Authentication is JWT-based for the local HTTP API.

The RAG pipeline is the most technically interesting piece. Extracted lab values and free-text segments are embedded with a local sentence-transformer model and stored in a vector index; at query time the assistant retrieves the user's own past readings and the relevant source-text chunks, then grounds its response in them and cites the document and page. This is what prevents the most dangerous failure mode — a confident, wrong medical statement.

3.3 Privacy posture

HealthCentral never uploads patient data. The model runs locally; the database is encrypted at rest; the HTTP API binds to localhost only. Third-party services (embedding model downloads, web search for medical reference terms during development) are sandboxed so that they never

see patient data. The design principle is that a compromise of the machine is required to leak anything, and a compromise of the cloud is architecturally impossible because there is no cloud.

4. The Research Thread: Why Frameworks Matter

The reason this report dwells on frameworks is that the choice of tools matters. Four months ago, HealthCentral could not have been built in its current form by a me alone. The reason it now can is that the surrounding ecosystem crossed a threshold: models became fluent enough, harnesses became reliable enough, and the vocabulary for how to work with them became sharp enough, to let one person orchestrate what would previously have been a small team.

The rest of this report is organized around that ecosystem, then returns to the project to show what changed inside HealthCentral at each step.

5. Frameworks, Tools, and Techniques Studied

5.1 Prompting evolution: vibe coding → meta-prompting → context engineering → harness engineering

The terminology that engineers use to describe AI-assisted coding shifted three times in eighteen months. In February 2025, Andrej Karpathy coined “vibe coding” on X: “fully give in to the vibes, embrace exponentials, and forget that the code even exists.” The phrase was a description of how experienced engineers were already working — offloading the mechanical parts of implementation and spending their attention on intent and outcome. It went viral and was still in use in Cal Poly Pomona’s CS4800 under Dr. Yu Sun as a way of framing how students should work with AI tools.

Meta-prompting is the more rigorous academic counterpart. Suzgun and Kalai’s January 2024 paper (arXiv:2401.12954) showed that a single language model can act as a conductor, generating tailored prompts for specialized sub-instances and then synthesizing their outputs. In practice, this is what Claude Code now does when it builds sub-agent prompts on the fly, injects ``<system-reminder>`` tags mid-conversation, and composes tool descriptions conditionally based on the environment.

Through mid-2025 the community settled on “context engineering” as the right label for the discipline one layer up: figuring out what the model sees at each step of a long-running task. LangChain published a guide, Cognition’s Devin team argued that single-agent systems with excellent context engineering beat most multi-agent architectures, and Karpathy reframed the discipline as “filling the context window with just the right information for the next step.”

By late 2025 the frontier moved one step higher. Anthropic engineering, HumanLayer, and commentators like Martin Fowler began writing about “harness engineering”: the infrastructure of tools, state management, permission governance, feedback loops, and recovery logic that turns a capable model into a capable worker. The shorthand equation — Agent = Model +

Harness — captures where the engineering discipline now lives. HealthCentral’s own development harness (project-specific CLAUDE.md files, hook-based secret scanning, a task tracker integrated with the planner subagent) is an instance of this.

5.2 Claude Code primitives: Skills, Subagents, Hooks, Slash Commands, Plugins, Agent SDK

Anthropic shipped a small set of composable primitives across 2025 that together made the agentic coding workflow usable.

Agent Skills (launched October 16, 2025; spec open-sourced December 2025) are folder-based capability packs. Each skill is a directory with a SKILL.md file that has YAML frontmatter and procedural instructions, plus optional scripts and resources. Skills load through progressive disclosure — the model sees only the frontmatter until the skill is invoked — which solves the “system prompt is bounded by the context window” problem by pushing knowledge onto the filesystem.

Subagents (mid-2025) let the main agent delegate work to spawned agents with their own isolated context windows, system prompts, and tool permissions. They roughly 3× speed up research-heavy tasks because they parallelize and keep the parent context clean.

Hooks (June 2025) are deterministic shell commands (or HTTP endpoints, or LLM prompts, or spawned subagents) that fire on lifecycle events: `SessionStart`, `UserPromptSubmit`, `PreToolUse`, `PostToolUse`, `Stop`. They are where teams add non-negotiable enforcement — linting, secret scanning, test gating, audit logging — without bolting it into prompts.

Slash commands are user-invoked Markdown files in `.claude/commands/`. They differ from skills in *who* invokes them: commands are explicit and user-triggered, skills are model-triggered based on context relevance.

Plugins (public beta October 9, 2025) bundle any combination of skills, subagents, hooks, MCP servers, and commands into a single installable unit. Marketplaces are decentralized — any Git repository can serve as one. This is what turned the customization layer from “power-user configuration” into a real ecosystem.

Claude Agent SDK (originally “Claude Code SDK”, renamed September 29, 2025) externalizes the harness so engineers can build production agents without re-implementing the tool loop.

5.3 Autonomous loops: Ralph, GSD v1/v2, Superpowers, everything-claude-code, NemoClaw/OpenClaw

Ralph’s loop (Geoffrey Huntley) is the canonical autonomous pattern. At its simplest: `while :; do cat PROMPT.md | claude -p ...; done`. The name is a Simpsons reference — Ralph Wiggum as an emblem of cheerful, persistent stubbornness. Huntley’s argument is that with a well-written prompt file and a real test gate, a surprisingly dumb loop can carry an agent through work an engineer would otherwise babysit.

GSD — "Get Shit Done" is the community evolution of Ralph. GSD v1 is a collection of prompts and skills dropped into `~/ .claude/commands/`. GSD v2 adds programmatic orchestration: it spawns a fresh subagent per sub-task (so each gets a clean context window), manages git branches, tracks tokens and cost, detects stuck loops, and auto-advances milestones. Where Ralph is the pattern, GSD is the pattern plus state management and spec-driven planning.

Superpowers (Jesse Vincent, `obra/superpowers`) is a plugin bundle built around a TDD-first methodology: no production code without a failing test first. It ships brainstorming skills, git-worktree workflows, and language-specific reviewer subagents. `everything-claude-code` (`affaan-m/everything-claude-code`) is the large-scale reference: dozens of subagents, many dozens of skills, and command shims across backend, frontend, and framework-specific stacks.

NemoClaw and OpenClaw are community experiments in agent-sandbox infrastructure that surfaced in spring 2026. They are mentioned here as *reference points* for what it might look like to run an entire test and build pipeline inside an agent-controlled sandbox; the specifics are still evolving and this report does not claim definitive knowledge of their architecture.

5.4 MCP and the tool ecosystem

Model Context Protocol (MCP), announced by Anthropic on November 25, 2024, is the open client/server protocol that standardizes how AI assistants connect to tools and data. Python and TypeScript SDKs shipped at launch alongside pre-built servers for Google Drive, Slack, GitHub, Git, Postgres, and Puppeteer. The March 2025 spec revision added Streamable HTTP transport. The ecosystem expanded fast: by the time HealthCentral development was underway in early 2026, the useful servers for this project already existed and were production-quality.

Supabase MCP exposes roughly twenty tools for database design, migration, and querying — used here for local Postgres schema evolution. **Vercel MCP** (remote, OAuth-authenticated, at `mcp.vercel.com`) was used for preview-deployment visibility. **Serena** (`oraios/serena`) is the one most worth dwelling on: it replaces text-based search with Language Server Protocol-powered semantic navigation so agents can reason about function definitions, cross-file references, and type relationships instead of greppy line matches. For a codebase that crosses Python and TypeScript like HealthCentral, Serena saved hours of context-window waste. **Tavily MCP** provides real-time web search and extraction, which was critical for medical-terminology cross-referencing where training data might be stale.

Google Stitch (Google Labs, announced at I/O 2025) turns text prompts or screenshots into responsive HTML/CSS, and exports to Figma. In HealthCentral it was used for first-pass UI scaffolding — dashboard layouts, biomarker trend cards, and the chat surface — which were then brought into React/TypeScript and refined with Claude Code and Codex.

5.5 Models

The models that drove the work were Anthropic’s Claude family — Opus 4.5 through Opus 4.7 and Sonnet 4.6 — and OpenAI’s GPT-5 Codex line (5.2 to 5.5 current), which the community referenced as 5.2, 5.3, 5.4, and now 5.5 through the winter and spring. The exact sub-version naming across the OpenAI side is not fully reflected in stable documentation; this report uses those names as the labels the community was using during each phase of the project rather than as canonical version metadata. The practical pattern was: Opus for planning and architecture, Sonnet for implementation, and Codex as the cross-vendor adversarial reviewer so that disagreements between the two families surfaced real bugs.

6. Chronological Timeline

The table below maps external developments to concrete changes in HealthCentral. It is the spine of this report: every row is a moment where something new appeared in the world and something measurably changed in how the project was built.

Phase	Date	External development	Impact on HealthCentral
0 — Baseline	Jan 2024	Suzgun & Kalai publish Meta-Prompting paper (arXiv:2401.12954)	Pre-project; established that LMs can orchestrate other LMs.
1 — Vibe coding	Feb 2025	Karpathy coins "vibe coding"; CS4800 (Dr. Yu Sun) uses it pedagogically	Project started as single-prompt iteration with Claude and GPT.
2 — MCP appears	Nov 2024 – early 2025	MCP spec + first servers released by Anthropic	Early connection of local Postgres via MCP for schema design.
3 — Meta-prompting in practice	Spring 2025	Meta-prompting migrates from paper to practice in Claude Code	Prompts became multi-stage: plan → critique → implement.
4 — Context engineering	Mid 2025	Karpathy, Cognition, LangChain articulate the discipline	RAG design budgeted tokens per source; older sessions summarized.
5 — Hooks + Subagents	Jun–Aug 2025	Claude Code hooks (Jun) and subagents (mid-2025) launch	Adversarial review split: Opus planner, Sonnet implementer, reviewer subagent.
6 — Skills	Oct 16, 2025	Anthropic launches Agent Skills (progressive disclosure)	Per-project SKILL.md folders for PDF ingestion, analyte mapping, citations.
7 — Plugins	Oct 9, 2025 (beta)	Claude Code plugin system + decentralized marketplaces	Adopted superpowers and everything-claude-code plugins.

Phase	Date	External development	Impact on HealthCentral
8 — Harness engineering named	Late 2025 – early 2026	Anthropic + HumanLayer formalize the term	Internal refactor: explicit CLAUDE.md, task-tracking, hook-based secret scanning.
9 — Ralph & GSD	Late 2025 – Q1 2026	Huntley's Ralph loop; GSD v1 → v2 community patterns	Overnight autonomous loops against a locked test suite; fresh-context subagents per task.
10 — Model upgrades	Nov 2025 – Feb 2026	Claude Opus 4.5 → Opus 4.6; Sonnet 4.6	Planning moved to Opus 4.6; implementation to Sonnet 4.6.
11 — Codex convergence	Q4 2025 – Q1 2026	GPT-5 Codex; community-reported 5.2 / 5.3 / 5.4 variants	Cross-vendor adversarial review: Claude writes, Codex reviews (and vice versa).
12 — NemoClaw / OpenClaw	Spring 2026	Community agent-sandbox experiments	Studied as references for agentic CI/build pipelines.
13 — Google Stitch	Mid-late 2025	Stitch moves from demo to usable tool	First-pass UI generation; reduced time-to-first-frontend-render.
14 — Submission	Apr 22, 2026	CS4610 final deliverables	Demo video + repo + this report.

A short prose narrative: the project started as vibe-coding to create a mock-up project and hopefully expand with experiences gained over-time. I described what I wanted and took whatever the model produced. Meta-prompting taught me to make the model critique its own plan before writing code. Context engineering taught me that the model's blind spot was almost never its intelligence, it was what it had last seen and available within each model's context window. Harness engineering taught me that the model was only one piece of a larger system, and that tools, state, and feedback loops were where quality came from. Throughout model's developments by both companies (OpenAI, Anthropic, Google, and a little bit of Qwen models through Cursor). By the end, HealthCentral was being built by a coordinated set of agents operating on a continuous improved harness.

7. How These Tools Shaped HealthCentral

7.1 Adversarial multi-model review

The most valuable workflow that emerged was cross-vendor adversarial review. Claude Opus would plan a change. Claude Sonnet would implement it. A Codex reviewer would read the diff

against the original plan and score it; Claude would then review Codex's review. When both sides agreed the change was good, it landed. When they disagreed, I resolved the disagreement. This caught several classes of bug that neither vendor alone would have caught: subtly wrong pandas indexing, a vector-search threshold that passed unit tests but produced garbage retrieval in practice, and a JWT leak in a log line.

7.2 Skills and hooks as privacy enforcement

HealthCentral's privacy guarantees patient data never leaves the device is too important to leave to prompt instructions. We encoded it into the harness. A `PreToolUse` hook scans for patterns that look like PHI and blocks any tool call that would send them to a network. A Skill for PDF ingestion contains the exact analyte-synonym map and unit-conversion rules, so the extraction process is the same every time rather than re-derived each session. This is an example of the harness doing work the prompt should not be trusted with.

7.3 Autonomous loops for RAG evaluation sweeps

RAG quality is a multi-dimensional problem: chunk size, embedding model, retrieval threshold, reranking, citation style. A Ralph-style autonomous loop ran overnight across a matrix of configurations, each evaluated against a locked set of held-out patient-question/answer pairs, and summarized the Pareto frontier by morning. This is work that would previously have required either weeks of iteration or a full-time ML engineer — a concrete example of agentic tools doing what they actually promise.

8. Reflections and Lessons

Three things worked that I would do again. First, writing down a clear `CLAUDE.md` at the start of every phase. It forced me to articulate what “done” meant, and every agent that read it performed better. Second, using a planner-implementer-reviewer split across three different models. The diversity between Opus, Sonnet, and Codex was worth the extra orchestration cost. Third, trusting the skills and hooks layer over prompt instructions for anything that was load-bearing. Prompts can be forgotten; hooks cannot.

Two things did not work as well. Running Ralph loops too long without a tight exit criterion cost more in tokens than it was worth; the lesson is that “autonomous” only pays off when the loop has a real stop condition. And leaning too heavily on Stitch-generated frontend produced code that looked right but did not match our accessibility targets; the fix was to treat Stitch output as a sketch, not a deliverable, and to finish every component by hand or with a second AI pass.

The deeper lesson is about the shape of software-engineering work. Writing code is no longer where the leverage is. The leverage is in stating the problem clearly, designing the harness that will solve it, and exercising good taste on the outputs. Agentic tools did not replace engineering; they moved where the engineering happens.

9. Deliverables

Source repository: github.com/BrooklynD23/HealthCentral

Demo video: accompanying the submission form; demonstrates PDF ingestion, biomarker trend view, and grounded-citation Q&A.

Planning document: 00_PLANNING_DOC.md (research compendium supporting this report).

Final report: this document.

10. References

- Karpathy, A. “Vibe coding” post, X, Feb 2, 2025. <https://x.com/karpathy/status/1886192184808149383>
- Suzgun, M., & Kalai, A.T. Meta-Prompting: Enhancing Language Models with Task-Agnostic Scaffolding, arXiv:2401.12954, Jan 2024. <https://arxiv.org/abs/2401.12954>
- Anthropic. Effective Harnesses for Long-Running Agents. <https://www.anthropic.com/engineering/effective-harnesses-for-long-running-agents>
- LangChain. Context Engineering for Agents. <https://blog.langchain.com/context-engineering-for-agents/>
- Anthropic. Equipping Agents for the Real World with Agent Skills. <https://www.anthropic.com/engineering/equipping-agents-for-the-real-world-with-agent-skills>
- Anthropic. Claude Code hooks reference. <https://docs.claude.com/en/docs/claude-code/hooks>
- Anthropic. Claude Agent SDK overview. <https://code.claude.com/docs/en/agent-sdk/overview>
- Huntley, G. Ralph’s loop. <https://ghuntley.com/ralph/>
- Vincent, J. obra/superpowers. <https://github.com/obra/superpowers>
- affaan-m. everything-claude-code. <https://github.com/affaan-m/everything-claude-code>
- Anthropic. Model Context Protocol announcement, Nov 25, 2024. <https://www.anthropic.com/news/model-context-protocol>
- Supabase. Supabase MCP Server. <https://supabase.com/blog/mcp-server>
- Vercel. Introducing Vercel MCP. <https://vercel.com/blog/introducing-vercel-mcp-connect-vercel-to-your-ai-tools>
- oraios/serena (semantic-retrieval MCP). <https://github.com/oraios/serena>
- Tavily MCP documentation. <https://docs.tavily.com/documentation/mcp>
- Google. Stitch UI design tool. <https://blog.google/products/google-labs/stitch-ui-design/>

Appendix A — Glossary

Agent harness: everything wrapping the model — tools, context, state, permissions, feedback loops. Agent = Model + Harness.

Context engineering: the discipline of curating what the model sees at each step of a long-running task.

Hook: a deterministic script fired on a lifecycle event (PreToolUse, PostToolUse, etc.) that can enforce or block behavior.

MCP: Model Context Protocol. Anthropic's open standard for connecting AI assistants to tools and data.

RAG: Retrieval-Augmented Generation. The assistant retrieves relevant source chunks before generating, and cites them.

Ralph loop: Geoffrey Huntley's pattern for running an agent in a simple shell loop until a task is complete.

Skill: a folder with a SKILL.md file; a reusable capability pack loaded by the agent through progressive disclosure.

Subagent: a spawned agent with its own context window, system prompt, and tool permissions.

Vibe coding: Karpathy's term for the informal, conversational style of directing an AI through implementation work.